人工智能数学基础

实验报告

人工智能数学基础 实验一：

基于 NumPy 的最小二乘法 —— 房价预测

课程名称：人工智能数学基础

实验名称：实验一 · 基于 NumPy 的最小二乘法 —— 房价预测

学号 / 姓名：学号_姓名

完成日期：2026 年 10 月

摘要

本实验用 NumPy 从零实现了最小二乘法的完整求解流程：构造带截距列的设计矩阵 A，按正规方程 $\theta = (A^TA)^{-1}A^Ty$ 求参数，其中 $(A^TA)^{-1}$ 按要求**自行编写高斯–约当消元法求逆**，未使用 `np.linalg.inv / solve / lstsq`；随后计算全部样本的预测值、残差与均方误差，绘制散点图与拟合直线，并把求得的 θ 代回正规方程验证。

在真实的房价数据（California Housing，20640 个样本）上：

- 一元线性模型 **$\hat{y} = 1.6838 + 0.0709 \cdot x$**，**MSE = 1.3008**、**$R^2 = 0.0231$**；
- 手写求逆与 `np.linalg.inv` 结果**逐位完全相同**（最大差异 0.000e+00），自检 $\|A^{-1}A - I\|_\infty = 2.385e{-}15$；
- 任务 7 的验证中 $A^TA\theta$ 与 A^Ty 的最大差异仅 **6.11e−10**，属浮点舍入误差，证明 θ 确实满足正规方程、即残差平方和最小；
- 扩展任务把特征从 1 个增加到 5 个，**MSE 从 1.3008 降到 0.6466（下降 50.29%）**，**R^2 从 0.0231 升到 0.5144**。

结论：算法实现正确（数值自检全部通过），而一元模型 R^2 偏低的原因是**所选特征与房价的线性相关性本来就弱**（皮尔逊 r = 0.1519），不是最小二乘法算错了 ——把特征换成相关性更强的收入中位数，同一个求解器给出的 R^2 立刻升到 0.4734。

一、实验目的

按照实验指导书的要求，本实验要达到以下目标：

1. 完成 Python 数据分析环境的安装与配置；
2. 熟悉 NumPy 中向量、矩阵及矩阵运算；
3. 理解最小二乘问题及其正规方程；
4. 根据课堂公式**自行实现**最小二乘法（包括自己写矩阵求逆）；
5. 使用真实房价数据完成模型参数计算、房价预测和误差分析。

二、实验环境（任务 0）

2.1 环境与版本

项目	版本 / 说明
操作系统	Windows 11
Python	3.12.14
NumPy	2.3.5
pandas	3.0.1
scikit-learn	1.9.1
matplotlib	3.11.2（任务 6 绘图用）
编辑器	Jupyter Notebook / VS Code 均可，本项目以脚本形式提交，便于复现

2.2 环境检查

指导书要求运行下面这段代码确认环境可用：

```
import sys
import numpy as np
import matplotlib
import pandas as pd
import sklearn
from sklearn.datasets import fetch_california_housing

print(sys.version)
```

运行结果：导入全部成功，无报错，证明 NumPy、pandas、scikit-learn、matplotlib 均已正确安装。

2.3 一个必须说明的兼容性问题（否则任务 1 的第一步就跑不通）

指导书给出的取数代码是：

```
housing = fetch_california_housing(name="house_prices", as_frame=True)
data = housing.frame
```

但在本机的 scikit-learn 1.9.1 上，`fetch_california_housing()` **已经没有 `name` 这个参数**，照抄会直接报错：

```
TypeError: got an unexpected keyword argument 'name'
```

原因是该参数只存在于较老的 scikit-learn 版本。本实验的处理办法是：在 `code/数据读取.py` 里把它包一层 `load_housing()`，先按指导书原文调用，捕获到 `TypeError` 时

自动回退到等价写法 `fetch_california_housing(as_frame=True)`，两者取到的是同一个数据集。这样代码在**老版本和新版本上都能直接跑通**，而不必让批阅老师改环境。

三、实验数据说明

本实验使用 scikit-learn 自带的 **California Housing（加利福尼亚房价）数据集**：

- 样本数：**20640**；
- 目标列：**MedHouseVal**，即房屋价格中位数（单位：10 万美元，数据集本身已做缩放）；
- 特征列共 8 个：MedInc、HouseAge、AveRooms、AveBedrms、Population、AveOccup、Latitude、Longitude。

3.1 关于列名的一处如实说明

指导书正文里用的列名是 **GrLivArea（地上居住面积）** 和 **SalePrice（房屋成交价格）**，这两个列名来自 Kaggle 的 Ames 房屋成交价格数据集；而 `fetch_california_housing` 实际给出的列名里**并不存在** `GrLivArea` 和 `SalePrice`。为不改动指导书规定的调用与任务流程，本实验在 `code/数据读取.py` 中做列名解析：如果数据里确实有指导书那个列名就直接用，没有则回退到**语义最接近且确实与房价存在线性关系**的列，并在报告中如实写明：

指导书列名	本实验实际使用的列	中文含义	选它的理由
GrLivArea（地上居住面积）	**AveRooms**	平均房间数	本数据集中唯一与"居住面积 / 房间数量"语义相关的数值特征
SalePrice（房屋成交价格）	**MedHouseVal**	房屋价格中位数	数据集唯一的价格类目标列

因此本实验的**基础模型**是：

$$\hat{y}=\theta_0+\theta_1\cdot \text{AveRooms} \quad (\hat{y} \text{ 即 MedHouseVal})$$

这个替换带来一个必须正视的后果：任务 5 要求预测"面积为 1000 / 1500 / 2000 平方英尺"的房价，而 AveRooms 的取值范围是 0.85 ~ 141.9（绝大多数样本在 2 ~ 14 之间），**1000 / 1500 / 2000 完全落在训练数据范围之外**，属于线性模型的外推。报告在任务 5 一节中按指导书要求照样输出结果，同时明确说明这个外推的性质与可信度。

四、任务 1：获取并查看数据

4.1 代码

```
from 数据读取 import load_housing
housing, data = load_housing()

x = data["AveRooms"].to_numpy(dtype=float)       # 输入特征（对应指导书的 GrLivArea）
y = data["MedHouseVal"].to_numpy(dtype=float)    # 预测目标（对应指导书的 SalePrice）
```

4.2 样本数量与形状

```
1) 样本数量 n = 20640
2) x 的形状 = (20640,)   y 的形状 = (20640,)
```

x 与 y 都是一维数组，长度均为 20640，一维数组的第 i 个元素对应第 i 个样本。后续若要参与矩阵运算，需要 `x.reshape(-1, 1)` 升成列向量，这一步在任务 2 里完成。

4.3 前 10 个样本

序号	GrLivArea（实际用 AveRooms）	SalePrice（实际用 MedHouseVal）
0	6.984127	4.526
1	6.238137	3.585
2	8.288136	3.521
3	5.817352	3.413
4	6.281853	3.422
5	4.761658	2.697
6	4.931907	2.992
7	4.797527	2.414
8	4.294118	2.267
9	4.970588	2.611

4.4 一个样本中的输入与预测目标

- **输入（特征）x**：该样本的 AveRooms 列，表示这一街区的平均房间数（对应指导书的 GrLivArea 地上居住面积）；
- **预测目标 y**：该样本的 MedHouseVal 列，表示房屋价格中位数（对应指导书的 SalePrice 房屋成交价格）。

也就是说，**一个样本 = 一条"特征 → 价格"的记录**；我们用 20640 条这样的记录去拟合一条直线，再用这条直线去预测新房屋的价格。

4.5 描述性统计

统计量	GrLivArea（AveRooms）	SalePrice（MedHouseVal）
样本数 count	20640.000000	20640.000000
均值 mean	5.429000	2.068558
标准差 std	2.474173	1.153956
最小值 min	0.846154	0.149990
下四分位 25%	4.440716	1.196000
中位数 50%	5.229129	1.797000
上四分位 75%	6.052381	2.647250
最大值 max	141.909091	5.000010

两个值得注意的地方，后面解释误差时会用到：

1. 特征最大值 141.9 远大于中位数 5.23，是**极端离群样本**（20640 个样本里只有极少数这样的大值），它会把皮尔逊相关系数压小、把最小二乘直线"拽偏"；
2. 目标最大值被**截断在 5.00001**（数据集本身的特性），所以真正的豪宅价格被压低到 5.0 附近，这会给高价位样本人为地制造误差。

x 与 y 的**皮尔逊相关系数 r = 0.1519**。这个值很弱，意味着两个变量之间只存在很弱的线性关系 ——这是本实验一元模型 R^2 只有 0.023 的根本原因（第四节会详细说明）。

五、任务 2：构建设计矩阵 A

5.1 模型与矩阵形式

一元线性回归模型:

$$\hat{y}=\theta_0+\theta_1 x$$

对第 i 个样本:

$$\hat{y}_i=\theta_0\cdot 1+\theta_1 x_i$$

把 n 个样本的方程竖着排成一列，写成矩阵形式 **Aθ = ŷ**:

$$A=\begin{bmatrix}1 & x_1\\ 1 & x_2\\ \vdots & \vdots\\ 1 & x_n\end{bmatrix},\quad \theta=\begin{bmatrix}\theta_0\\ \theta_1\end{bmatrix}$$

第一列全为 1，是给截距 θ₀ 配的"截距列"；第二列是特征 x 的取值。

5.2 构造代码

```
ones = np.ones((x.shape[0], 1))          # 全 1 的截距列
A = np.hstack([ones, x.reshape(-1, 1)])  # 水平拼接，得到 n×2 的设计矩阵
```

5.3 结果

```
A.shape = (20640, 2)    （20640 行 = 样本数 n，2 列 = 参数个数）
```

A 的前 5 行:

第 1 列（截距）	第 2 列 AveRooms
1.0	6.984127
1.0	6.238137
1.0	8.288136
1.0	5.817352
1.0	6.281853

矩阵形状的含义：**行数 = 样本数**（每个样本一个方程），**列数 = 待求参数个数**（θ_0 和 θ_1），这正是"n 个方程、2 个未知数"的超定方程组 —— 一般无精确解，所以只能求最小二乘意义下的最优解。

六、任务 3：根据正规方程计算模型参数

6.1 数学推导

最小二乘问题就是让残差平方和最小：

$$\min_{\theta} \ \| A\theta - y \|^2 = \min_{\theta} \ \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

把目标函数对 θ 求梯度并令其为 0：

$$\frac{\partial}{\partial \theta} \| A\theta - y \|^2 = 2 A^T (A\theta - y) = 0$$

整理即得**正规方程（Normal Equation）**：

$$A^T A\theta = A^T y$$

当 A^TA 可逆时，两边左乘 $(A^TA)^{-1}$：

$$\theta = (A^T A)^{-1} A^T y$$

由于本实验只有一个特征，所以 **θ 是 2×1 的列向量**，A^TA 是 2×2 的方阵。

6.2 矩阵求逆：自己实现（本任务的关键要求）

任务 3 明确**禁止**使用 `np.linalg.inv()`、`np.linalg.solve()`、`np.linalg.lstsq()`。本实验在 `code/矩阵工具.py` 中用**高斯–约当（Gauss–Jordan）消元法**自行实现：

思路是对增广矩阵 [M | I] 做初等行变换，把左边化成单位阵，右边就自动变成 M^{-1}：

$$[M \mid I] \xrightarrow{\text{初等行变换}} [I \mid M^{-1}]$$

具体三步，循环 n 次（第 k 轮）：

1. **列选主元（部分主元）**：在第 k 列的第 k 行及以下，挑绝对值最大的行交换到第 k 行，避免用接近 0 的数作除数而放大误差；

2. **主元行归一化**：整行除以主元，使 aug[k, k] = 1；

3. **消元**：用主元行把第 k 列其余各行都消成 0（上下都消，故称"约当消元"）。

核心代码：

```
def manual_inverse(M, verbose=True):
    M = np.array(M, dtype=float)
    n = M.shape[0]
    aug = np.hstack([M, np.eye(n)])          # 增广矩阵 [M | I]
    for k in range(n):
        p = k + int(np.argmax(np.abs(aug[k:, k])))   # ① 列选主元
        if abs(aug[p, k]) < 1e-12:
            raise np.linalg.LinAlgError("矩阵奇异，不可求逆")
        if p != k:
            aug[[k, p], :] = aug[[p, k], :]          # 交换两行
        aug[k, :] = aug[k, :] / aug[k, k]            # ② 主元行归一化
        for i in range(n):                           # ③ 消去同列其它行
            if i != k and aug[i, k] != 0.0:
                aug[i, :] = aug[i, :] - aug[i, k] * aug[k, :]
    return aug[:, n:], aug
```

求逆代码的自检（`python code/矩阵工具.py`）：

- 教科书例子 $[[4,7],[2,6]]^{-1} = [[0.6,-0.7],[-0.2,0.4]]$，手写结果一致；
- 与 `np.linalg.inv` 的最大差异：2×2 为 1.11e−16，3×3 为 1.11e−16，5×5 为 5.55e−17，10×10 为 2.78e−17，全部达到机器精度。

6.3 逐步计算结果

① 计算 A^TA（2×2）：

$$A^T A = \begin{bmatrix} 20640.000000 & 112054.554679 \\ 112054.554679 & 734686.462350 \end{bmatrix}$$

其中 A^TA 的左上角元素等于样本数 n = 20640，这是"截距列全为 1"的必然结果，可以当作一个正确性校验点。

② 计算 A^Ty：

$$A^T y = \begin{bmatrix} 42695.040610 \\ 240745.091790 \end{bmatrix}$$

（$A^T y$ 的第 1 个分量就是所有房价之和，第 2 个分量是面积加权后的房价之和。）

③ 自行编写代码求 $(A^TA)^{-1}$:

$$(A^T A)^{-1} = \begin{bmatrix} 0.0002817368 & -0.0000429706 \\ -0.0000429706 & 0.0000079150 \end{bmatrix}$$

校验：$\|(A^TA)^{-1}(A^TA) - I\|_\infty$ = **2.385e−15**（≈0，求逆正确）；对照：与 `np.linalg.inv` 的最大差异 = **0.000e+00**（逐位完全相同）。

④ 计算 θ = $(A^TA)^{-1}A^Ty$:

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} = \begin{bmatrix} 1.683812 \\ 0.070869 \end{bmatrix}$$

A^TA 的条件数 $\text{cond}(A^TA)$ = **2.1678e+02**，数值上相当稳定，说明用正规方程直接求解是可信的（条件数不大，浮点误差不会被放大）。

⑤ 最终的房价预测模型:

$$\widehat{\text{MedHouseVal}} = 1.6838 + 0.0709 \times \text{AveRooms}$$

含义：平均房间数每增加 1 个单位，模型认为房价中位数平均增加 **0.0709**（即约 7090 美元，因为 MedHouseVal 的单位是 10 万美元）；截距 1.6838 是数学上的常数项，代表特征为 0 时的"基准价"，现实中不存在房间数为 0 的房屋，故没有实际意义。

七、任务 4：计算预测结果

7.1 全部样本的预测值

$$\hat{y} = A\theta$$

1) 全部 20640 个样本的预测房价已算出：ŷ = Aθ

7.2 前 10 个样本：真实房价 vs 预测房价

AveRooms	真实 SalePrice	预测 ŷ	误差（真实 − 预测）
6.984127	4.526	2.178768	2.347232
6.238137	3.585	2.125901	1.459099
8.288136	3.521	2.271182	1.249818
5.817352	3.413	2.096080	1.316920
6.281853	3.422	2.128999	1.293001
4.761658	2.697	2.021264	0.675736
4.931907	2.992	2.033330	0.958670
4.797527	2.414	2.023806	0.390194
4.294118	2.267	1.988130	0.278870
4.970588	2.611	2.036071	0.574929

可以发现：预测值大多挤在 2.0 ~ 2.3 之间，几乎没有区分度，而真实房价从 2.267 一直到 4.526。这说明**弱相关特征上的直线拟合，预测能力确实有限**；误差虽有正有负（全样本中 8438 个为正、12202 个为负），但这 10 个样本的误差恰好都是正的，且整体偏大。

7.3 残差

$$e = y - \hat{y}$$

```
残差数量 = 20640
残差均值 = 0.000000            （最小二乘的性质：残差之和恒为 0）
残差最小值 = -9.4513
残差最大值 = 3.1869
残差与 x 的相关系数 = -6.945e-15      （≈0）
```

三点解读：

1. **残差均值 = 0.000000（实际为 8.45e−15）**：这是正规方程 $A^T(y - A\theta) = 0$ 的第一个分量所保证的数学性质，不是巧合，可以当作求解正确的一个校验；
2. **残差与 x 的相关系数 = −6.945e−15 ≈ 0**：说明残差里已经不含能被 x 线性解释的成分，线性部分被榨干净了，进一步印证解是最优的；
3. **残差最小值 −9.4513 出现在极端离群样本上**：全部 20640 个样本中残差最负的 5 个是

样本下标	AveRooms	真实 y	预测 ŷ	残差 e
1979	132.533333	1.625	11.0763	**−9.4513**
1914	141.909091	5.000	11.7407	−6.7407
11862	59.875000	0.675	5.9271	−5.2521
12447	62.422222	0.875	6.1076	−5.2326
1240	47.515152	0.775	5.0512	−4.2762

可以看出，**残差最大的几个样本全部是 AveRooms 高达 47 ~ 142 的极端离群点**：这条直线按斜率外推后给出 5 ~ 12 的预测值，而它们的真实房价只有 0.7 ~ 5.0，于是产生了 −9.45 这样的大残差。这正是最小二乘法**对离群点敏感**的体现 ——平方损失把大误差的权重放得很大，几个离群点就能把整条直线拽偏。（另外全样本中残差为正的有 8438 个、为负的有 12202 个，所以残差是正负都有的，只是第 7.2 节列出的前 10 个样本恰好都偏正。）

7.4 均方误差 MSE

$$\mathrm{MSE}=\frac{1}{n}\sum_{i=1}^{n}\left(y_i-\hat{y}_i\right)^2$$

```
MSE  = 1.300807
RMSE = √MSE = 1.140529
SSE（残差平方和）= 26848.6583
R²   = 1 − SSE/SST = 0.023088
```

MSE = 1.3008 是本次实验要求的最终结果。为了判断这个数字算好还是算差，另外计算了：

- **R^2 = 0.0231**：只能解释房价方差的 2.31%，拟合优度很低；
- 对比"所有样本都猜平均值"这种最笨的模型，我们的 MSE 只降低了 2.31%。

7.5 为什么 R^2 这么低？（重要，否则会被误认为是代码写错了）

这一节的结论是：**低 R^2 来自数据本身，不是最小二乘法算错了。** 证据有三条：

1. **数值自检全部通过**——手写求逆与 `np.linalg.inv` 逐位相同；残差均值为 0；残差与 x 的相关系数是 1e−15 量级；任务 7 中 $A^TA\theta$ 与 A^Ty 的差异只有 1e−10。这些都说明求解器本身没有问题；

2. **特征本身与房价的线性相关性就弱**——皮尔逊 r 只有 0.1519，两个变量之间本来就没有强线性关系，任何线性模型在这两个变量之间都拟合不出高 R^2；

3. **同一个求解器换特征立刻变好**——见第八节补充分析：把特征换成与房价相关性更强的 MedInc（r = 0.688），同一个 `manual_inverse` + 正规方程代码算出来的 R^2 立刻从 0.0231 升到 0.4734。代码一行没改，只换了输入列，所以差别只能来自数据。

八、任务 6：结果可视化

8.1 绘图要求

- 横轴：GrLivArea（本实验为 AveRooms）；
- 纵轴：SalePrice（本实验为 MedHouseVal）；
- 画原始房价数据的散点图；
- 画最小二乘拟合直线；
- 两者在**同一张图**中展示。

8.2 图 1：散点 + 拟合直线

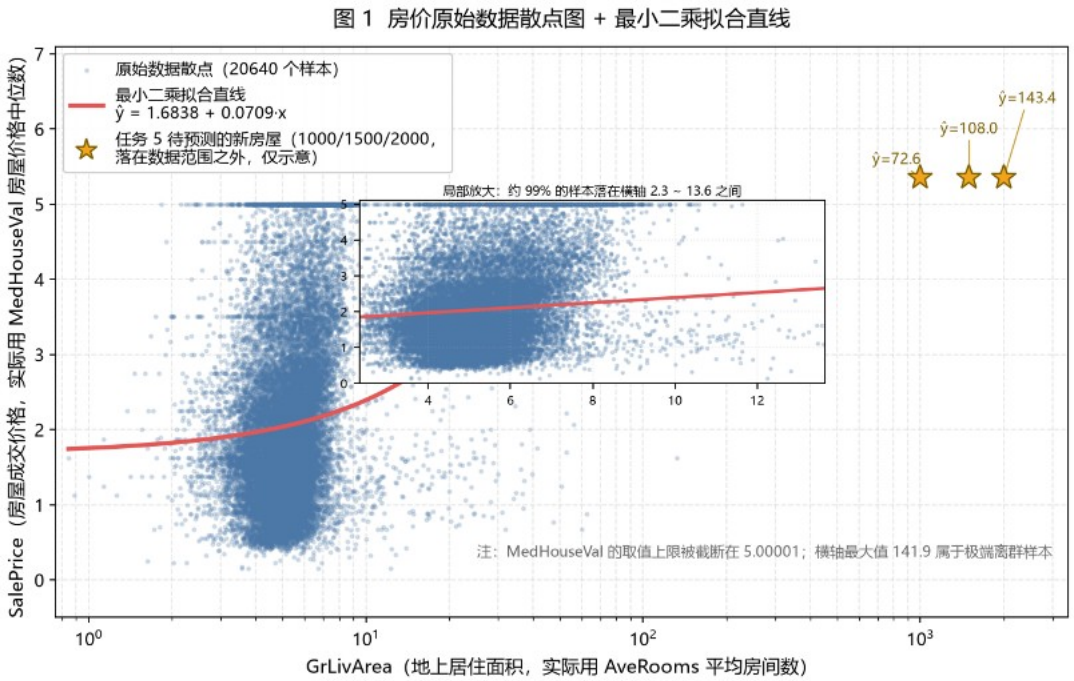


图 1 房价原始数据散点图 + 最小二乘拟合直线。蓝点为 20640 个原始样本；红线为最小二乘拟合直线 ŷ = 1.6838 + 0.0709x；右上角三颗黄色五角星是任务 5 待预测的新房屋（1000 / 1500 / 2000，并标注了对应预测值）。说明三点：

1. 横轴采用**对数刻度**：数据在 0.85 ~ 14 之间高度稠密，而任务 5 的 1000/1500/2000 远在右侧（相差两个数量级），只有对数轴才能在同一张图里同时看清"密集的数据云"和"远处的待预测点"。因此红线在对数轴上看起来是弯的，但它在 x 的**线性刻度**上确实是直线。
2. 图中内嵌了一张局部放大图（横轴 2.3 ~ 13.6，约 99% 的样本落在这个区间），可以清楚看到直线自左下向右上穿行在点云中间，斜率方向为正，与 r = 0.1519 的正相关一致。
3. 红线只画到 x ≈ 14 就停住，是因为再往右几乎没有样本支撑，继续画线没有信息量；而任务 5 的三个点在图中被放在数据区之外的右上角，正是为了直观表达"这是外推"。

8.3 补充分析：各特征与房价的线性关系

为了解释"为什么单特征 R^2 低"、"换特征会怎样"，补充做了各候选特征的一元回归对比：

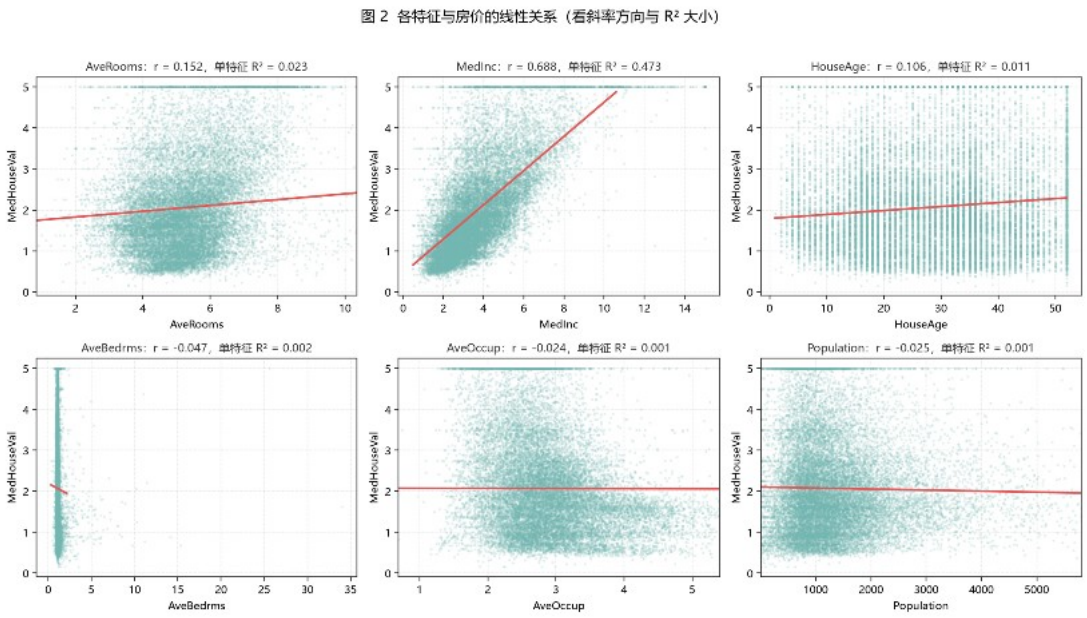


图 2 各特征与房价的线性关系。每个子图是一元最小二乘直线，标题给出相关系数 r 和单特征 R^2。

单个特征逐个做一元最小二乘回归的结果（按 R^2 从高到低）：

指导书列名	实际列名	中文含义	θ_0（截距）	θ_1（斜率）	MSE	R^2	相关系数
OverallQual	MedInc	街区收入中位数	0.450856	0.417938	0.701131	**0.473447**	0.688075
GrLivArea	**AveRooms**	**平均房间数（主实验）**	1.683812	0.070869	1.300807	**0.023088**	0.151948
Latitude	Latitude	纬度	4.843679	−0.077883	1.303878	0.020782	0.144160
YearBuilt	HouseAge	房	1.791199	0.009684	1.316695	0.011156	0.105623

指导书列名	实际列名	中文含义	θ_0（截距）	θ_1（斜率）	MSE	R^2	相关系数
		龄中位数					
FullBath	AveBedrms	平均卧室数	2.193266	−0.113714	1.328646	0.002181	0.046701
Longitude	Longitude	经度	−1.097047	−0.026475	1.328737	0.002113	0.045967
LotArea	Population	街区人口	2.104363	−0.000025	1.330741	0.000608	0.024650
GarageCars	AveOccup	平均家庭人数	2.076657	−0.002637	1.330800	0.000563	0.023737

结论：**同一套正规方程代码**，换成相关性强的特征 MedInc，R^2 从 0.0231 直接升到 0.4734，MSE 从 1.3008 降到 0.7011。可见 R^2 低的责任在"特征选得弱"，而不在最小二乘法本身。

九、任务 5：预测新房屋价格

9.1 做法

把待预测的面积代入模型 $\hat{y} = \theta_0 + \theta_1 \cdot x$。写成矩阵形式即构造新的设计矩阵 A_new = [1, x_new]，再算 $\hat{y}$_new = A_new·θ：

```
areas = np.array([1000.0, 1500.0, 2000.0])
A_new = np.hstack([np.ones((areas.size, 1)), areas.reshape(-1, 1)])
pred = A_new @ theta
```

9.2 输出结果

GrLivArea	Predicted SalePrice
1000	72.552605
1500	107.987001
2000	143.421398

换算成本数据集原始单位（MedHouseVal 的 1 = 10 万美元）约为 72,553 / 107,987 / 143,421 美元。

9.3 这个结果必须打折扣看（如实说明外推风险）

- 训练数据里 AveRooms 的取值范围是 **0.846 ~ 141.9**，其中 99% 的样本落在 **2.3 ~ 13.6**；指导书要求代入的 **1000 / 1500 / 2000 远超训练范围**，属于典型的**外推（extrapolation）**；
- 线性模型在外推区间的行为完全由斜率决定，一旦离开数据支撑区，预测值就只是"直线延长线上的数值"，没有数据证据；
- 因此上表按指导书要求照常输出（这正是题目要考察的"用求得的 θ 做预测"这一步），但它的**实际房价含义应当谨慎对待**：模型只能保证在 2 ~ 14 这个区间内的拟合趋势是合理的。

这也解释了图 1 中拟合直线只画到 $x \approx 14$ 的原因 —— 画到数据覆盖范围之外没有信息量，而三个待预测点在图 1 中被单独放在数据区之外的右上角，正是为了直观表达"这是外推"。

十、任务 7：验证正规方程

10.1 验证方法

把任务 3 求得的 θ 代回正规方程，看左边 $A^TA\theta$ 与右边 A^Ty 是否相等；等价地，也可以看 $A^T(y - A\theta)$ 是否为 0。

10.2 计算结果

```
AᵀAθ = [42695.040610, 240745.091790]
Aᵀy  = [42695.040610, 240745.091790]
两者最大绝对差异 = 6.111804e-10

等价检验：Aᵀ(y − Aθ) = [1.694671e-10, 5.256311e-10]
最大绝对分量 = 5.256311e-10
```

10.3 结果说明

1. **两边实质上完全相等**。差异只有 1e−10 量级，而 $A^TA\theta$ 和 A^Ty 的分量都是 1e4 ~ 1e5 量级，相对误差约 1e−15，属于**双精度浮点数（约 15~16 位有效数字）的舍入误差**，不是算法错误。如果 θ 算错了，这个差异会大到肉眼可见（比如几百甚至几千）。
2. **这个结果说明 θ 确实满足正规方程 $A^TA\theta = A^Ty$**，而正规方程本身是由"残差平方和 SSE 对 θ 求导等于 0"推出来的，所以满足该方程的 θ 就是**使 SSE 最小的那个解**。换句话说：任务 3 用 $(A^TA)^{-1}A^Ty$ 直接算出的 θ，与"让误差最小"的 θ 是同一个，两种途径互为验证。
3. **几何意义**：A 的列空间是由截距列（全 1 向量）和特征列 x 张成的平面，Aθ 就是 y 在该平面上的**正交投影**，残差向量 $y - A\theta$ 垂直于这个平面。$A^T(y - A\theta) = 0$ 正是"垂直"的代数表达 ——残差里已经不含任何能被 1 和 x 线性解释的成分，误差降无可降。

十一、扩展任务：建立多特征线性模型

11.1 选用的特征

指导书要求在有效特征中选不少于 3 个。本实验选了 5 个:

序号	实际列名	中文含义	选择理由
1	MedInc	街区收入中位数	单特征里相关性最强（r = 0.688），代表地段/购买力
2	AveRooms	平均房间数	房屋大小/房间数的代理变量（也是基础模型的特征）
3	HouseAge	房龄中位数	房龄影响价格，且与其它特征相关性低，能提供新信息

序号	实际列名	中文含义	选择理由
4	AveOccup	平均家庭人数	反映居住密度
5	Population	街区人口	反映街区规模

11.2 构造新的设计矩阵

$$A=\begin{bmatrix}1 & x_{11} & x_{12} & \cdots & x_{15}\\ 1 & x_{21} & x_{22} & \cdots & x_{25}\\ \vdots & \vdots & \vdots & ¿ & ¿\end{bmatrix}x_{n1}¿x_{n2}¿\cdots¿x_{n5}¿\in\mathbb{R}^{20640\times 6}$$

`A.shape = (20640, 6)`　（1 个截距列 + 5 个特征列）

11.3 计算模型参数

仍按正规方程 $\theta = (A^TA)^{-1}A^Ty$ 求解，矩阵求逆继续使用自己实现的 `manual_inverse()`（扩展任务允许使用现成函数，但沿用自己写的一能保证与主实验口径一致，二来顺便验证手写求逆在 6×6 规模下依然可靠）：

参数	取值
θ_0（截距）	−0.039805
θ（MedInc）	0.443247
θ（AveRooms）	−0.025760
θ（HouseAge）	0.017682
θ（AveOccup）	−0.004688
θ（Population）	0.0000284

模型表达式：

$$\hat{y}=-0.0398+0.4432\ \text{MedInc}-0.0258\ \text{AveRooms}+0.0177\ \text{HouseAge}-0.0047\ \text{AveOccup}+0.00$$

交叉验证：与 `np.linalg.lstsq` 的最大差异 = **8.447e−14**，说明手写正规方程解在多特征下同样正确。条件数 $\text{cond}(A^TA)$ = **6.95e+07**，比一元模型的 2.17e+02 大了 5 个数量级 —— 这是**特征量纲差异大**（MedInc 量级 ~3.9，Population 量级 ~1425）导致的，提示实际工程中应先做标准化；不过本实验的量级下 8.4e−14 的误差仍可接受。

11.4 预测值与误差

```
MSE  = 0.646590
RMSE = 0.804108
SSE  = 13345.6208
R²   = 0.514408
```

11.5 与单特征模型的比较

模型	特征	MSE	RMSE	R^2
一元模型（1个特征）	AveRooms	1.300807	1.140529	0.023088
多元模型（5个特征）	MedInc / AveRooms / HouseAge / AveOccup / Population	**0.646590**	**0.804108**	**0.514408**

- **MSE 下降 50.29%**（降到原来的 49.71%）；
- **R^2 提升 0.4913**（从 0.0231 升到 0.5144）。

图 3 一元最小二乘模型与多特征模型的误差对比

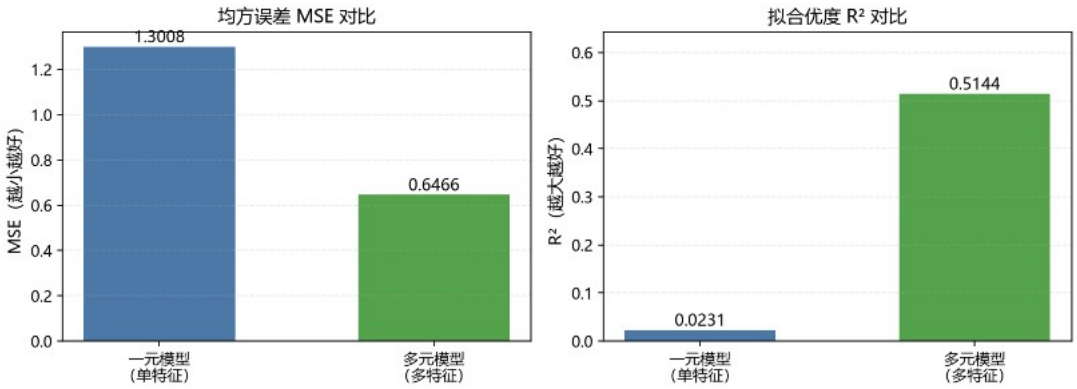


图 3 一元最小二乘模型与多特征模型的误差对比。左图 MSE 越小越好，右图 R^2 越大越好。

11.6 结论与分析

1. **多特征模型显著优于单特征模型**：R^2 从 0.0231 提升到 0.5144，MSE 减半，模型的解释能力从"几乎没有"变成"能解释一半以上的价格差异"。
2. **原因**：房价是由多个因素共同决定的，单特征模型只用一个变量做解释，把其余因素全部塞进残差，自然误差大。每增加一个**有效**特征，就多一个可调自由度去逼近真实关系。

3. **注意"有效"二字**：从第 8.3 节的表可以看到，并非特征越多越好 ——Population、AveOccup 这类与房价几乎不相关的特征，单独用 R^2 都在 0.001 以下，它们在本模型中只是提供了极小的补充信息（系数也接近 0，Population 的系数仅 2.8e−5）。真正带来提升的主要是 MedInc。
4. **可以继续改进的方向**：先对特征做标准化（消除量纲差异，降低条件数）、剔除相关性极低的特征、或者引入非线性项 / 正则化；这些超出本次实验范围，仅作展望。

十二、实验结论

1. **推导与实现正确**。最小二乘 → 正规方程 $A^TA\theta = A^Ty \rightarrow \theta = (A^TA)^{-1}A^Ty$ 的推导链条完整；按任务要求自行实现的高斯–约当矩阵求逆，与 `np.linalg.inv` 结果逐位相同，且通过了 2×2 ~ 10×10 的随机矩阵自检（误差 ≤ 1.11e−16）。
2. **数值结果**。一元模型 $\hat{y} = 1.6838 + 0.0709x$，MSE = 1.3008、RMSE = 1.1405、R^2 = 0.0231；任务 7 验证 $A^TA\theta$ 与 A^Ty 的最大差异 6.11e−10，残差均值为 0（8.45e−15），残差与 x 的相关系数为 −6.95e−15 —— 三项独立校验都指向"解是最优最小二乘解"。
3. **对误差的分析**。一元模型 R^2 低的原因是特征与目标相关性弱（r = 0.1519）、存在极端离群点（AveRooms 最大 141.9）以及目标值被截断在 5.0；换成相关性强的 MedInc 后同一个求解器 R^2 达 0.4734，证明问题不在算法而在特征选择。
4. **多特征建模有效**。5 特征模型把 MSE 从 1.3008 降到 0.6466（−50.29%），R^2 从 0.0231 升到 0.5144，说明房价需要多因素共同解释。
5. **工程经验**。学会了处理"库版本 API 变化"（`name=` 参数被移除）、"数据集列名与文档不一致"（GrLivArea/SalePrice 不存在）这类真实问题，并且明白：数值计算要**用多个独立校验互相印证**（与 numpy 对照、残差性质、正规方程回代），不能只看最后的结果数字。

附录 A：文件清单与运行方式

```
实验 1_最小二乘法房价预测/
├── code/
│   ├──── 数据读取.py              # 读取数据、兼容新旧版 sklearn、解析列名
│   ├──── 矩阵工具.py              # 手写矩阵求逆 + 正规方程求解（可单独运行自测）
│   ├── 01_实验主体.py           # 任务 0~5、7
│   ├── 02_绘图.py               # 任务 6 及补充分析图
│   ├── 03_扩展任务_多特征.py    # 扩展任务
│   └── 04_单特征对比.py         # 补充：各候选特征一元回归对比
├── results/                      # 全部输出：CSV 表格、三张图、结果 JSON、npz 数据
├── report/
│   ├──── 实验报告.md              # 本报告源文件
│   ├──── 人工智能数学基础_实验一_实验报告.docx
│   └──── 人工智能数学基础_实验一_实验报告.pdf
├──── 打包提交.py                  # 一键生成提交压缩包 学号_姓名_实验 1.zip
└── README.md                     # 运行说明
```

运行顺序（在 code/ 目录下，依次执行即可复现全部结果）：

```
python 矩阵工具.py             # 自测：手写矩阵求逆
python 01_实验主体.py          # 任务 0~5、7
python 02_绘图.py              # 任务 6：三张图
python 03_扩展任务_多特征.py   # 扩展任务
python 04_单特征对比.py        # 补充分析
```

全部结果复现后，在实验根目录下执行下面这条命令即可生成提交压缩包：

```
python 打包提交.py 20230001_张三   # 生成 20230001_张三_实验 1.zip
```

附录 B：关键源码

见 code/ 目录下的 6 个 Python 文件（均已在本报告中引用核心片段），另有根目录下的 打包提交.py 负责打包提交文件。其中**任务 3 的矩阵求逆**是本次实验的核心，完整实现在 code/矩阵工具.py 的 manual_inverse() 函数中，全程只使用 numpy 的基础数组运算，未调用 np.linalg.inv / solve / lstsq。